

Chapter 6

CS1 course package - Webster University

Xiaoyuan Suo¹ and Peter Maher¹

¹Webster University

PDC course material used in CS1 at Webster University

Abstract

This chapter presents a course transformation in an introductory computer science sequence (CS1) at Webster University, which is a primarily undergraduate institution, where Parallel and Distributed Computing (PDC) concepts are integrated without requiring prior background in systems or advanced mathematics. The target audience includes STEM undergraduates with little to no programming experience. The primary learning outcomes are for students to (1) understand core PDC ideas such as parallelism, speedup, and efficiency, (2) reason about workload distribution and performance trade-offs, and (3) connect conceptual understanding to simple implementations in C++. The transformation emphasizes accessible, low-barrier entry points through a combination of brief interactive visualizations, selective unplugged activities, and lightweight programming exercises, enabling students to grasp foundational performance concepts early in the curriculum.

Descriptor Elements

We designed a sequence of activities that progressively build students' understanding of PDC concepts, moving from intuitive, unplugged experiences to guided visualizations and finally to lightweight programming tasks.

Programming Language Employed: C++

Relevant core courses: CS1

Pre-requisites: no prerequisites

Textbook Starting Out with C++ from Control Structures to Objects, 10th edition, Tony Gaddis [3]

Relevant PDC topics and Bloom's taxonomy: Classification of PDC topics according to the revised Bloom's Taxonomy [1]: Remember (RE), Understand (UN), Apply (AP), Analyze (AN), Evaluate (EV), and Create (CR).

Learning outcomes:

1. understand core PDC ideas such as parallelism, speedup, and efficiency,
2. reason about workload distribution and performance trade-offs, and
3. connect conceptual understanding to simple implementations in C++.

Table 6.1: PDC topics and their corresponding levels in the revised Bloom’s Taxonomy.

PDC Topic	Bloom’s Level
Introduction to Parallel Computing	RE
Task Decomposition and Partitioning	UN
Speedup and Efficiency	AN
Performance Measurement and Benchmarking	AP

Context for use: Webster University is a primarily undergraduate institution located in St. Louis. The Department of Computer and Information Sciences offers several undergraduate programs, including a B.S. in Computer Science with multiple emphasis areas. Introductory programming instruction is delivered through a two-course CS1 sequence consisting of COSC 1550 and COSC 1560. These courses introduce students to fundamental concepts in C++ programming, including variables, control structures, functions, arrays, pointers, file I/O, abstract data types (ADTs), basic object-oriented programming, and dynamic memory management. In addition to programming syntax and problem-solving skills, students develop foundational computational thinking skills through hands-on programming assignments and laboratory activities. Typical enrollment in each course ranges from approximately 12 to 25 students per semester.

6.1 Introduction

At Webster University, we have explored ways to introduce Parallel and Distributed Computing (PDC) concepts early in the curriculum, specifically within a CS1 course taught in C++. While PDC is traditionally reserved for upper-level courses, delaying exposure often reinforces the misconception that parallelism is an advanced or niche topic. In reality, parallel thinking, such as breaking problems into independent tasks, reasoning about concurrency, and understanding performance trade-offs, is a fundamental computational skill [6]. Introducing these ideas in CS1 helps students develop a more accurate mental model of modern computing, where multicore processors and parallel workloads are the norm rather than the exception.

Our approach integrates PDC concepts in the later part of the semester (around week 13 of a 16-week course), after students have developed a solid foundation in loops, arrays, and functions. At this stage, students are ready to revisit familiar problems through a new lens: not just how to solve them, but how to solve them efficiently using parallelism. We focus on two carefully chosen examples: an embarrassingly parallel summation problem and a parallel search algorithm. These examples allow students to see immediate connections between sequential and parallel implementations without introducing unnecessary complexity.

6.2 How CS1 was changed and PDC topics integrated

We modified several traditional CS1 topics, particularly loops and arrays, to incorporate introductory parallel and distributed computing (PDC) concepts without significantly

changing the overall structure of the course. Loops were used to introduce ideas such as data parallelism, independent task execution, and workload decomposition by demonstrating how repetitive operations can be divided among multiple workers. Arrays provided a natural context for discussing partitioning large datasets into smaller segments that can be processed concurrently. Simple examples such as searching, counting, and grid-based simulations were adapted to help students recognize opportunities for parallel execution while still reinforcing core programming concepts.

6.2.1 Integrated Syllabi (Pre and Post)

The overall structure of the CS1 sequence remained largely unchanged, allowing the integration of parallel and distributed computing (PDC) concepts without removing core introductory programming content. Instead, selected topics were enhanced with parallel thinking activities, demonstrations, and programming exercises. Traditional CS1 concepts such as loops, arrays, searching, and simulations were used as natural entry points for introducing decomposition, concurrency, workload partitioning, and performance considerations.

Table 6.4 (in the appendix) summarizes the major course topics and highlights where PDC-related modifications were incorporated.

6.2.2 Highlights

Core CS1 content remained largely unchanged, including variables, control structures, functions, arrays, pointers, object-oriented programming, and debugging. The primary changes involved embedding PDC concepts into existing topics through activities, simulations, and performance-oriented examples. Some highly language-specific or less essential STL-focused material was reduced or simplified to create instructional time for the new content.

Unplugged Activities

Before adopting animations, we relied on unplugged activities to introduce core PDC ideas without programming syntax [4]. One commonly used exercise was the *coin search* activity, where a group of students is asked to find a specific coin in a large jar. The task is first performed by a single student (sequential search), and then repeated with multiple students searching simultaneously (parallel search). This simple setup helps students experience, in a tangible way, how dividing work can reduce completion time, while also revealing limitations such as uneven workload distribution or coordination overhead.

We also used a *flag coloring* 3.3.1 activity[5], where students are given a map or flag divided into regions and asked to color it under certain constraints (e.g., no adjacent regions sharing the same color). Students work either individually or in groups, with each person responsible for different regions, simulating task decomposition and concurrent execution. While the activity reinforces ideas such as independence of tasks and the need for coordination, many students perceived it as a time-consuming task as it usually takes a whole class period. Based on this feedback, we began using short animations that convey the same concepts more efficiently, allowing us to move more quickly from intuition to implementation.

Timing and Teaching Tips The coin-search activity typically requires 15–20 minutes, while the full flag-coloring activity may require 30–50 minutes depending on class size and discussion. Before beginning, ask students to predict whether doubling the number of workers will halve the completion time. Afterward, discuss uneven workload distribution, coordination costs, and why measured speedup may differ from the ideal case. For shorter class periods, the corresponding animations can be used instead of the full unplugged activities.

Programming Demo

After establishing these conceptual foundations, we introduce OpenMP as a practical tool for parallel programming in C++. However, this step presents its own challenges. Many students struggle with the syntax of compiler directives, particularly the meaning and role of `#pragma`. Rather than treating it as a purely syntactic feature, we frame `#pragma omp` as an instruction to the compiler that enables parallel execution of code regions. By tying this back to the earlier animations and activities, students can interpret OpenMP not as “magic,” but as a mechanism for expressing the parallel ideas they already understand conceptually.

In the programming demo shown in Table 6.2, students generated vectors of random integer values and, using our provided code, compared sequential and parallel implementations of the summation operation. Because each array element can be processed independently, the example provided a simple introduction to data parallelism and reduction operations while remaining accessible to CS1 students.

Table 6.2: Sequential and parallel versions of summing values in an array.

Sequential Version	Parallel Version
<pre> int sum_serial(const vector<int>& data) { int sum = 0; for (int i = 0; i < data.size(); ++i) { sum += data[i]; } return sum; } </pre>	<pre> int sum_parallel(const vector<int>& data) { int sum = 0; #pragma omp parallel for reduction(+ : sum) for (int i = 0; i < data.size(); ++i) { sum += data[i]; } return sum; } </pre>

Listing 6.1 shows a modified bubble-sort example using OpenMP. Instead of comparing every adjacent pair sequentially, the algorithm alternates between even- and odd-indexed pairs, enabling independent comparisons to be executed in parallel.

Listing 6.1: OpenMP odd-even parallel Odd-Even sort example

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <omp.h> (e.g., -fopenmp)

// Parallel Odd-Even Sort implementation
void parallelOddEvenSort(std::vector<int>& arr) {
    int n = arr.size();

    // The algorithm requires up to n total phases to guarantee sorting
    for (int phase = 0; phase < n; ++phase) {

        // Phase 0, 2, 4... -> Even Phase (pairs: 0-1, 2-3, 4-5...)
        // Phase 1, 3, 5... -> Odd Phase (pairs: 1-2, 3-4, 5-6...)
        if (phase % 2 == 0) {
            // Parallelize the even-indexed pairs loop
            #pragma omp parallel for shared(arr, n)
            for (int i = 0; i < n - 1; i += 2) {
                if (arr[i] > arr[i + 1]) {
                    std::swap(arr[i], arr[i + 1]);
                }
            }
        } else {
            // Parallelize the odd-indexed pairs loop
            #pragma omp parallel for shared(arr, n)
            for (int i = 1; i < n - 1; i += 2) {
                if (arr[i] > arr[i + 1]) {
                    std::swap(arr[i], arr[i + 1]);
                }
            }
        }
    }
}

int main() {
    // Sample unsorted array
    // Speed up will not show because the data size,
    // should use a much bigger data size
    std::vector<int> data = {34, 7, 23, 32, 5, 62, 78, 4, 0, 11, 99, 14};

    std::cout << "Original_Array:";
    for (int num : data) std::cout << num << ",";
    std::cout << "\n";

    // Run parallel sort
    parallelOddEvenSort(data);

    std::cout << "Sorted_Array:";
    for (int num : data) std::cout << num << ",";
    std::cout << "\n";

    return 0;
}

```

Timing and Teaching Tips The parallel-sum demonstration typically requires 20–25 minutes, and the odd-even sort demonstration requires approximately 25–30 minutes. Begin with the familiar sequential version and ask students to identify which operations are independent before showing the OpenMP code. Use a sufficiently large input when discussing performance, since small examples may run more slowly in parallel because thread-creation and synchronization overhead dominate the computation.

6.2.3 Challenges

One of the primary challenges in integrating PDC concepts into CS1 was the limited amount of available instructional time. Introductory programming courses are already densely packed with foundational topics such as variables, control structures, functions, arrays, pointers, object-oriented programming, and debugging. To incorporate PDC material without overwhelming students, we modernized existing topics rather than adding entirely new units. Concepts such as data decomposition, concurrency, and workload partitioning were embedded into familiar topics including loops, arrays, and searching. To create space for these additions, some highly language-specific or less commonly used STL-focused content was reduced.

Another pedagogical challenge involved balancing conceptual understanding with programming complexity. Since many CS1 students are novice programmers, introducing advanced ideas such as synchronization, scalability, and communication overhead too early can increase cognitive load. To address this issue, we relied heavily on unplugged activities and animations as visual demonstrations before introducing code-based examples. Activities such as coin-search exercises, zombie outbreak simulations, and image-processing demonstrations helped students develop intuition about parallel execution in an accessible way.

Designing effective programming assignments also became increasingly difficult due to the widespread availability of generative AI tools. Traditional introductory programming assignments can often be solved directly using AI-generated code, making it harder to evaluate genuine student understanding. As a result, we tried to redesign assignments to emphasize experimentation and debugging, but the efforts still need to be further evaluated.

At the same time, because many students had not yet taken courses in computer architecture or operating systems, advanced performance concepts had to be presented conceptually rather than mathematically. The instructional approach therefore emphasized intuition-building and empirical experimentation using lightweight tools and small-scale examples that could run on standard student laptops without requiring specialized hardware.

6.3 New Unplugged Activities

6.3.1 Animations and Visualizations

A key challenge in teaching PDC at this level is abstraction. Concepts such as threads, synchronization, and compiler directives can be difficult for beginners to grasp, especially when introduced purely through code. To address this, we incorporate visual and interactive teaching strategies. First, we use lightweight animations (Parallel Search Animation)

and (Parallel Flag Coloring Animation Figure 6.1) to illustrate how tasks are divided and executed concurrently. These animations help students build an intuitive understanding of parallel execution, such as how multiple “workers” operate simultaneously, how workload is distributed, and why certain problems scale better than others. By visualizing execution, students can more easily connect code behavior with underlying computational processes.

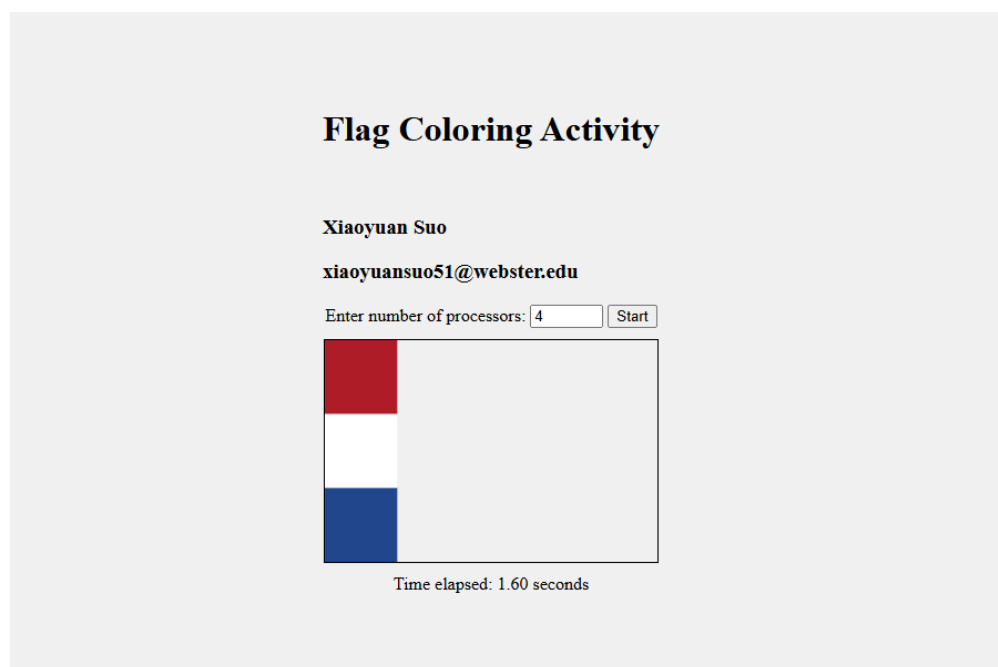


Figure 6.1: Flag coloring animation

Timing and Teaching Tips Each animation can be used in approximately 10–15 minutes. Ask students to predict the next step or identify which tasks can execute concurrently before running the animation. Pause at key points to discuss task decomposition, dependencies, synchronization, and workload balance. The animations are most effective when immediately connected to a familiar CS1 example, such as array searching, loops, or a simple OpenMP demonstration.

6.4 New Plugged-in Activities

6.4.1 Zombie Attack Assignment

As for the assignment, students complete a “*zombie attack*” homework problem[2]. In this assignment, students model the spread of a zombie infection across a grid-based environment and investigate how the simulation can be decomposed and executed in parallel. The task is intentionally designed to be conceptually approachable while exposing students to fundamental PDC ideas, including data partitioning, locality, synchronization, and scalability. Students are encouraged to experiment with different decomposition strategies (e.g., dividing the grid into subregions) and to reason about communication overhead and performance

trade-offs as the number of processing units increases. This assignment serves as a bridge between conceptual understanding and computational thinking, reinforcing key ideas in a creative and engaging context; see Figure 6.2.

To support this activity, I developed a lightweight animation that visually demonstrates the infection propagation process: (Zombie Attack Animation). To clarify, the animation illustrates the spread using a single zombie; however, in the actual assignment, students work with multiple zombies initialized at random positions on a significantly larger grid. This extension increases the complexity of the simulation and provides a more realistic setting for exploring parallel execution and performance considerations.

Zombie Outbreak Simulation

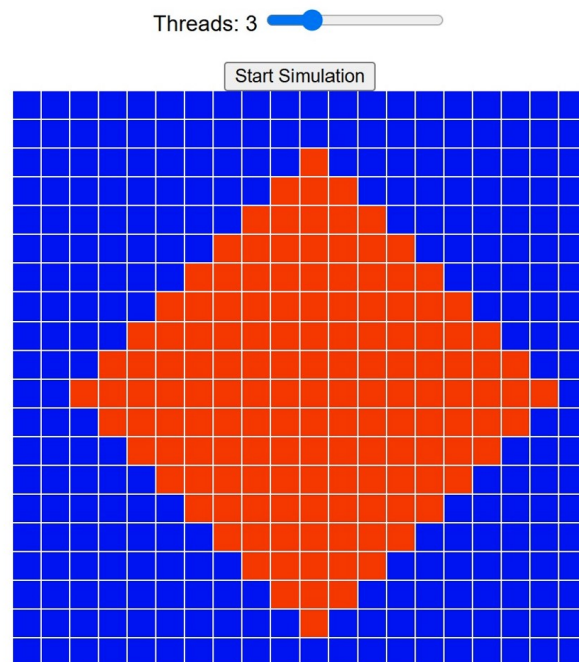


Figure 6.2: Zombie Attack homework: Red grid represents Zombie and Blue grid represents unaffected human

Timing and Teaching Tips Introduce the animation in 10–15 minutes before assigning the programming task, which typically requires one to two weeks. Begin with a small grid and one zombie so students can verify each update manually, then increase the grid size and number of zombies. Emphasize correctness before performance, and ask students to identify boundary cells, shared data, and the points at which synchronization is required before they begin parallelizing the simulation.

6.5 Course-wide Pre and Post Course Surveys

A detailed data analysis can be found in the appendix. The survey results indicate measurable gains in both PDC-related familiarity and core programming skills. Across the three PDC-related experience items, the composite mean increased from 1.62 on the pre-survey to 2.21 on the post-survey. Favorable responses (ratings of 5 or higher) increased from 0% to 25% for parallel processing, distributed computing, and event handling. Although students remained novice learners in PDC, the results suggest that the course successfully introduced concepts that were largely unfamiliar at the beginning of the semester.

Larger gains were observed in foundational C++ programming topics that support later PDC learning. The composite programming-experience mean across ten major C++ topics increased from 4.17 to 5.26. The strongest improvements were observed for structures ($\Delta M = 2.73$), classes and objects ($\Delta M = 1.93$), arithmetic expressions ($\Delta M = 1.09$), pointers ($\Delta M = 0.98$), and arrays ($\Delta M = 0.80$). These gains are particularly relevant because arrays, pointers, and structured data are heavily used in the parallel programming activities and simulations introduced in the course.

Student feedback regarding instructional activities was also highly positive. Programming assignments received the highest post-survey rating ($M = 6.62$), with 100% of students rating them favorably. Attending class ($M = 6.50$) and in-class practice activities ($M = 6.12$) were also rated highly. These findings suggest that the combination of unplugged activities, visual animations, and hands-on programming assignments was effective in supporting student learning and engagement with introductory PDC concepts.

6.6 Other Activities and Ideas

Table 6.3 summarizes the plugged and unplugged activities used to introduce PDC concepts in my CS1 course, along with their approximate instructional time. Not all activities are used in every semester; the selection depends on the course schedule and instructional priorities. In practice, animations are used more frequently than unplugged activities because they convey the same concepts while requiring less class time. The reported durations are approximate and may vary depending on the level of student interaction, discussion, and the extent to which the instructor expands the activities with hands-on exercises and demonstrations.

Support Community One important future direction is the development of a broader community of instructors adopting introductory PDC teaching modules. A shared community could support the exchange of assignments, animations, classroom activities, datasets, and teaching experiences, particularly for faculty at teaching-focused institutions who may not have formal training in parallel computing.

Using Animations Unplugged activities are often effective for introducing abstract ideas such as concurrency, synchronization, and workload decomposition. However, many of these activities can feel overly simplistic or time-consuming for CS1 or CS2. Visual animations may provide a useful middle ground by retaining the intuitive nature of unplugged activities

Table 6.3: Summary of PDC activities integrated into the CS1 course.

Activity	Approx. Time	Week	Relevant PDC Topics	Relevant CS1 Topics
Coin Search (Unplugged)	20 min	8	Sequential vs. parallel execution, workload decomposition, task independence	Linear search, loops
Flag Coloring Animation	30 min	8	Task decomposition, concurrency, synchronization, dependency	Problem solving
Parallel Search Animation	10 min	8	Data partitioning, embarrassingly parallel problems	Arrays, searching, loops
Parallel Sum Demonstration (OpenMP)	25 min	14	Data parallelism, reduction, compiler directives, speedup	Arrays, loops, functions
Odd-Even Parallel Sort Demo	30 min	14	Synchronization, independent tasks, parallel algorithms	Sorting, arrays, functions
Zombie Attack Assignment	1–2 weeks	15–16	Domain decomposition, scalability, locality, performance reasoning	2D arrays, simulation, nested loops, functions
Performance Discussion	15 min	16	Execution time, benchmarking, speedup, efficiency	Algorithm analysis, timing

while allowing instructors to introduce concepts more efficiently during class. Developing a shared repository of lightweight educational animations could therefore be valuable to the broader PDC education community. The author also plans to continue developing additional interactive animations and visual teaching materials.

The Need of a Textbook Survey responses suggest that students are interested in modern computing topics that extend beyond traditional introductory programming curricula. Topics such as parallelism, simulations, and high-performance computing appeared to increase student engagement because they exposed students to contemporary computing challenges and applications. At the same time, early undergraduate students still require strong foundational instruction and structured textbook support before they can meaningfully engage with more advanced PDC concepts.

Prerequisite is missing introducing PDC concepts in CS1/CS2 courses remains challenging because many students lack the mathematical maturity and programming background typically associated with traditional parallel computing courses. Some PDC topics also do not naturally align with the programming languages commonly used in introductory courses. For example, while C++ is effective for teaching memory management, arrays, and performance-oriented programming, it provides relatively limited support for event-driven programming compared to languages and frameworks designed specifically for asynchronous or reactive systems.

6.7 Conclusion

The modules mentioned above required minimal mathematical background and were designed to fit naturally within existing CS1/CS2 curricula without major restructuring. Student feedback and assessment results suggest that approachable, application-driven examples can increase interest and confidence in PDC topics at an early stage of the curriculum. Future work includes expanding the collection of animations and reusable teaching modules and exploring additional ways to integrate modern performance-oriented computing topics into introductory programming education.

Bibliography

- [1] Lorin W Anderson and David R Krathwohl. A taxonomy for learning, teaching, and assessing: A revision of Bloom's taxonomy of educational objectives: complete edition. Addison Wesley Longman, Inc., 2001.
- [2] H. Martin Buckner, Johannes Schoder, Xiaoyuan Suo, and D. Bunde. Peachy parallel assignments. In Proceedings of EduPar-25: 15th NSF/TCPP Workshop on Parallel and Distributed Computing Education, Milan, Italy, June 2025. Held in conjunction with the 39th IEEE International Parallel & Distributed Processing Symposium (IPDPS).
- [3] Tony Gaddis. Starting Out with C++ from Control Structures to Objects. Pearson, 10th edition, 2023. Pearson eText platform.
- [4] Suzanne J Matthews. Pdcunplugged: A free repository of unplugged parallel distributed computing activities. In 2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pages 284–291. IEEE, 2020.
- [5] Mary Smith, Srishti Srivastava, David P Bunde, April Crockett, Michael Gerten, Peter Maher, Jaime Spacco, Xiaoyuan Suo, Jiayin Wang, and Michelle Zhu. A visual unplugged activity to introduce pdc. In 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pages 658–665. IEEE, 2025.
- [6] Xiaoyuan Suo, Olga Glebova, David Liu, Alina Lazar, and Doina Bein. A survey of teaching pdc content in undergraduate curriculum. In 2021 IEEE 11th Annual Computing and Communication Workshop and Conference (CCWC), pages 1306–1312. IEEE, 2021.

Appendix

Github Link to Instructional Material

<https://github.com/xiaoyuansuo51-webster/teaching-PDC-HPC>

Detailed Pre and Post Syllabus

Table 6.4: High-level syllabus with PDC enhancements in CS1.

Traditional CS1 Topic	PDC-Related Enhancement
Introduction to Programming	Introduced the concept of parallel computing through unplugged activities and real-world examples of concurrency. (concepts only)
Loops and Iteration	Discussed independent iterations, data parallelism, and dividing repetitive tasks among multiple workers. (concepts, programming demo)
Arrays and Strings	Demonstrated data decomposition and partitioning of large datasets into smaller independent sections. (concepts, programming demo)
Searching and Counting Problems	Compared sequential and parallel approaches using classroom activities and simple performance measurements. (concepts, programming demo)
Grid-Based Simulations	Used zombie spread and image-processing style activities to illustrate parallel updates on large datasets. (concepts, programming demo, programming assignments)
Performance Measurement	Introduced basic timing experiments, speedup, and efficiency using small C++ examples. (in class demo)
Final Programming Assignments (Zombie Attack)	Included parallel-thinking exercises emphasizing decomposition, scalability, and workload distribution.

Organization of Material

GitHub Resources and Teaching Materials

The instructional materials, animations, and programming assignments used in this module are publicly available on my GitHub.

- **Parallel Search Animation**

Parallel Search Animation

This animation demonstrates the concept of parallel search by visually comparing how multiple workers can search simultaneously across a problem space. The activity helps students understand workload decomposition and the motivation for parallel execution.

- **Zombie Attack Simulation Animation**

Zombie Attack Simulation

This animation illustrates a simplified grid-based zombie outbreak simulation used to introduce parallel simulations and data decomposition. In the visualization, red cells represent zombies and blue cells represent humans. The animation intentionally uses only a single zombie to simplify the visualization before students transition to the full programming assignment involving large grids and multiple zombies.

- **Flag Coloring Activity**

Flag Coloring Activity

This activity uses the Netherlands flag as a simple visual model for discussing parallelism and synchronization. The three horizontal stripes were intentionally chosen to allow students to reason about dividing work into independent regions that can be processed concurrently.

- **Parallel Linked List Activity**

Parallel Linked List Animation

This animation demonstrates that although linked lists are inherently sequential data structures and cannot be easily traversed in parallel, different operations or tasks associated with the linked list can still be executed concurrently. The activity helps students distinguish between data-level parallelism and task-level parallelism.

- **Complete Teaching Repository**

Teaching-PDC-HPC Repository

This repository contains the instructional materials used throughout the module, including animations, assignments, and supporting input data files. Key materials include:

- `ZombieAttack_XSU0.html`: Interactive animation introducing the zombie outbreak simulation concept.
- `Zombie_Assignment_V2.docx`: Parallel programming assignment in C++ using OpenMP.
- `zombie_input_files.zip`: Sample input datasets for large-scale zombie outbreak simulations.

Table 6.5: Pre/post changes in PDC-related experience. Percent favorable is the percentage of responses coded 5 or higher.

Construct	Pre M	Post M	ΔM	Pre %	Post %	Δ pp	d
Parallel processing	1.50	2.25	0.75	0.0	25.0	25.0	0.54
Distributed computing	1.75	2.12	0.38	0.0	25.0	25.0	0.26
Event handling	1.62	2.25	0.62	0.0	25.0	25.0	0.43

Survey and Other Anonymized Data and Analysis

Survey responses were analyzed using a seven-point ordinal scale. For experience items, responses were coded from 1 = no experience to 7 = extremely experienced. For agreement items, responses were coded from 1 = strongly disagree to 7 = strongly agree. Percent favorable was calculated as the percentage of students selecting 5 or higher on the seven-point scale. Because the pre- and post-surveys were anonymous and analyzed as aggregate responses, the results are reported as group-level pre/post changes rather than matched individual gains.

Overall, the data show that students entered the course with relatively strong general programming confidence but limited exposure to parallel and distributed computing (PDC). The composite mean across the three PDC-related experience items increased from 1.62 on the pre-survey to 2.21 on the post-survey, a gain of 0.58 points on the seven-point scale. The percentage of favorable responses increased from 0% to 25% for parallel processing, distributed computing, and event handling. These results suggest that the course introduced students to PDC concepts that were largely unfamiliar at the beginning of the semester.

The strongest quantitative gains occurred in core COSC 1560 programming topics that support later PDC learning. The composite programming-experience mean across ten C++ topics increased from 4.17 to 5.26, for an average gain of 1.08 points. The largest individual gains were observed for C++ structures ($\Delta M = 2.73$), classes and objects ($\Delta M = 1.93$), arithmetic expressions ($\Delta M = 1.09$), pointers ($\Delta M = 0.98$), branching ($\Delta M = 0.84$), and arrays ($\Delta M = 0.80$). These results are important because arrays, pointers, structures, and object-oriented design provide the technical foundation for later examples involving data decomposition, memory organization, and task-based problem solving.

Student attitudes toward computing remained generally positive. In the post-survey, 87.5% of students rated their interest in computing as favorable, and 87.5% rated their general understanding of computing as favorable. The PDC-specific attitudinal measures were more conservative: 50.0% gave favorable ratings for interest in PDC, 50.0% gave favorable ratings for the importance of PDC, and 37.5% gave favorable ratings for understanding of PDC. This pattern is consistent with a course design in which PDC is introduced as an embedded CS1/CS2-level topic rather than as a full dedicated parallel computing course; students gained initial exposure, but their self-reported mastery remained lower than their general programming confidence.

Perceived helpfulness of course activities. Post-survey responses also indicate that students viewed the main instructional activities as helpful. Programming assignments re-

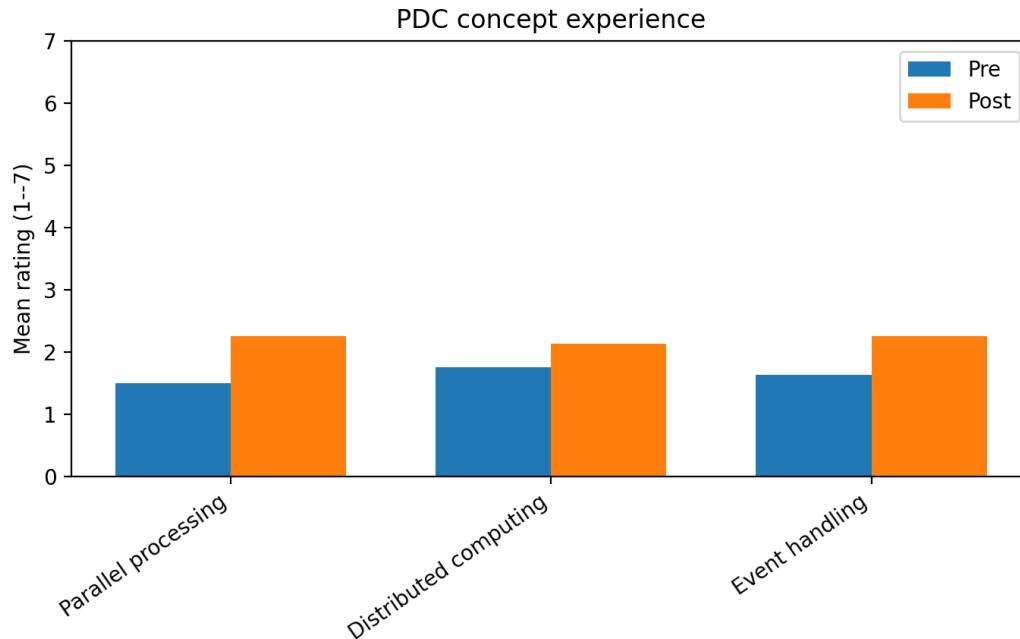


Figure 6.3: Pre/post comparison of student self-reported experience with PDC-related concepts. Mean ratings are reported on a seven-point scale.

ceived the highest rating ($M = 6.62$), with 100% of responses favorable and 100% rated very helpful or extremely helpful. Attending class was similarly strong ($M = 6.50$), followed by in-class practices ($M = 6.12$), textbook use ($M = 5.62$), exam preparation ($M = 5.62$), and meeting with the instructor outside of class ($M = 5.62$). These results support the use of a layered instructional sequence: direct instruction, in-class practice, programming assignments, and assessment preparation.

Post-course learning outcomes. Students reported high confidence on the major COSC 1560 learning outcomes. All students gave favorable ratings for writing C++ programs using arrays, pointers, structured data, and file operations; designing programs using department style guidelines; understanding the array–pointer relationship; reading and writing text/binary files; and using constructors and destructors. The strongest post-course outcome means were for arrays/pointers/structures/files ($M = 6.12$), department style guidelines ($M = 6.00$), and array–pointer relationships ($M = 6.00$). These outcomes align well with the PDC module because PDC examples depend on students’ ability to reason about data layout, repeated operations over collections, and decomposition of work into smaller units.

The survey results indicate that students showed measurable gains in PDC-related familiarity and larger gains in the programming foundations needed to understand PDC examples. The post-survey activity ratings further suggest that students benefited most from active and practice-oriented components, especially programming assignments, class attendance, and in-class practice.

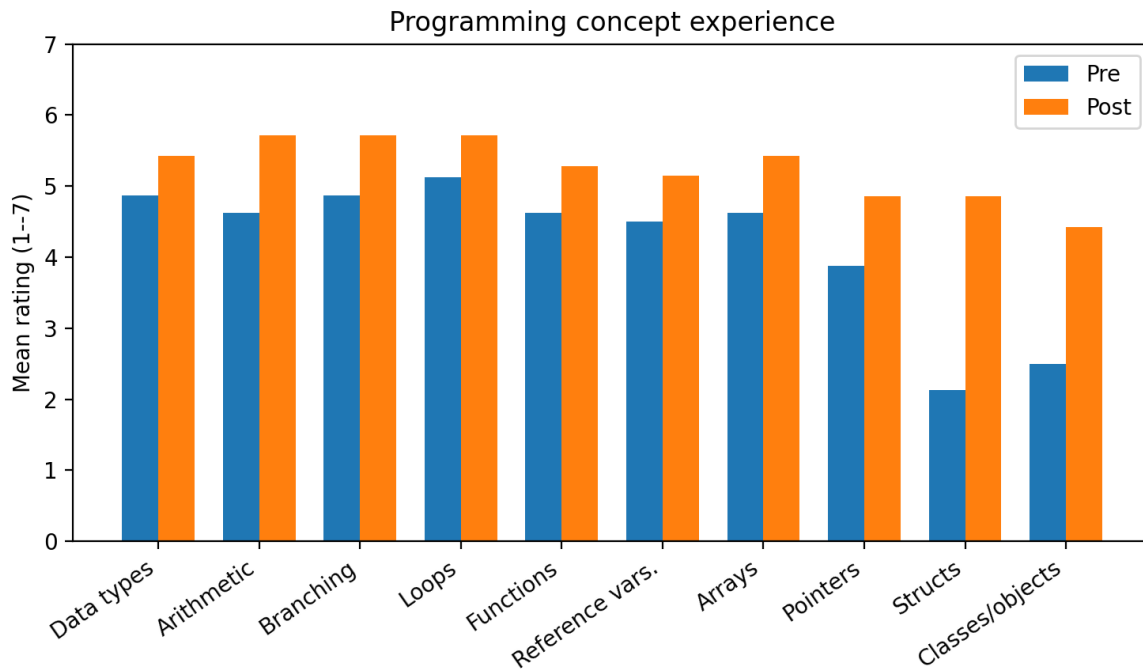


Figure 6.4: Pre/post comparison of student self-reported programming experience across core C++ topics.

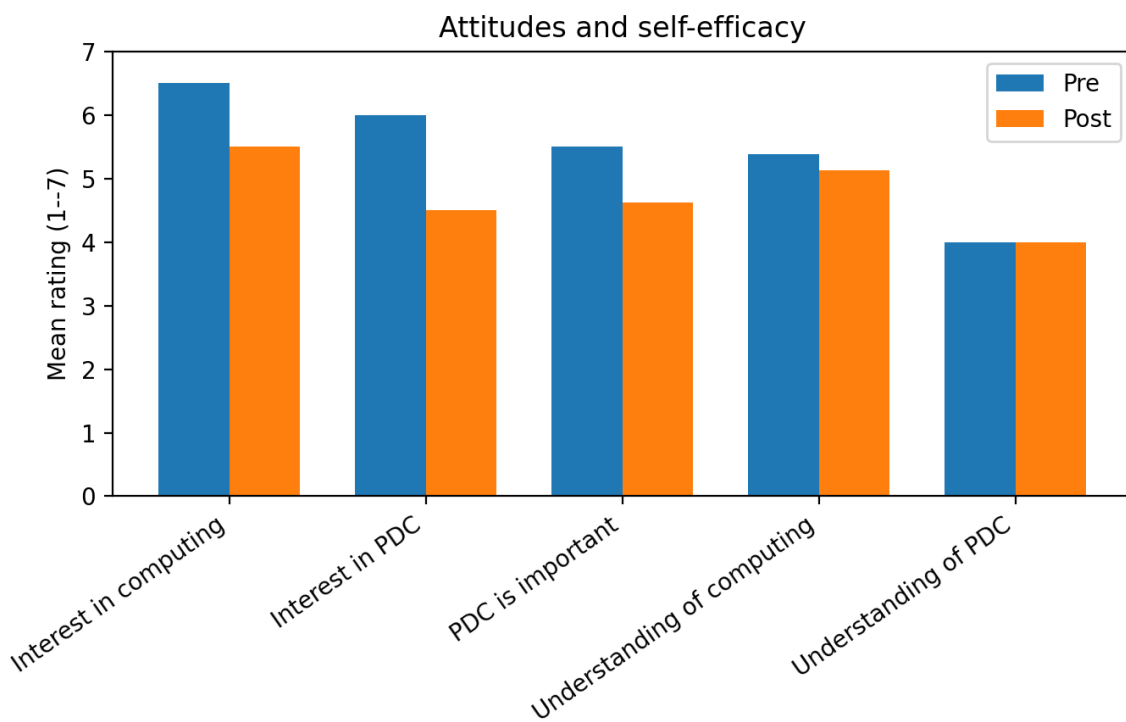


Figure 6.5: Pre/post comparison of student attitudes, interest, and self-reported understanding.

Table 6.6: Pre/post changes in programming experience across core C++ concepts.

Construct	Pre M	Post M	ΔM	Pre %	Post %	Δ pp	d
Data types	4.88	5.43	0.55	62.5	85.7	23.2	0.52
Arithmetic	4.62	5.71	1.09	62.5	85.7	23.2	0.81
Branching	4.88	5.71	0.84	75.0	85.7	10.7	0.80
Loops	5.12	5.71	0.59	87.5	100.0	12.5	0.66
Functions	4.62	5.29	0.66	62.5	85.7	23.2	0.78
Reference vars.	4.50	5.14	0.64	62.5	85.7	23.2	0.89
Arrays	4.62	5.43	0.80	75.0	71.4	-3.6	0.85
Pointers	3.88	4.86	0.98	37.5	57.1	19.6	0.76
Structs	2.12	4.86	2.73	12.5	57.1	44.6	2.22
Classes/objects	2.50	4.43	1.93	25.0	42.9	17.9	1.28

Table 6.7: Pre/post changes in interest, perceived importance, and self-reported understanding.

Construct	Pre M	Post M	ΔM	Pre %	Post %	Δ pp	d
Interest in computing	6.50	5.50	-1.00	100.0	87.5	-12.5	-0.88
Interest in PDC	6.00	4.50	-1.50	100.0	50.0	-50.0	-1.20
PDC is important	5.50	4.62	-0.88	75.0	50.0	-25.0	-0.64
Understanding of computing	5.38	5.12	-0.25	87.5	87.5	0.0	-0.26
Understanding of PDC	4.00	4.00	0.00	37.5	37.5	0.0	0.00

Table 6.8: Post-survey ratings of course learning activities.

Activity	Mean	Favorable %	Very/Extremely Helpful %
Textbook	5.62	100.0	50.0
Programming assignments	6.62	100.0	100.0
In-class practices	6.12	100.0	87.5
Attending classes	6.50	100.0	100.0
Meeting with the instructor outside of class	5.62	62.5	62.5
Tutoring provided by REEG ARC peer tutors	4.38	25.0	12.5
Exams and preparing for exams	5.62	100.0	50.0

Table 6.9: Post-survey ratings of course learning outcomes.

Learning outcome	Mean	Favorable %	Agree/Strongly Agree %
Arrays/pointers/structs/files	6.12	100.0	87.5
Department style guidelines	6.00	100.0	62.5
Array–pointer relationship	6.00	100.0	75.0
Character/string functions	5.38	87.5	50.0
Text/binary files	5.88	100.0	75.0
ADTs and structures	5.25	62.5	50.0
Classes with public/private members	5.62	87.5	62.5
Constructors/destructors	5.62	100.0	50.0